Computational Complexity

Reading guide and Exercises for **Week 01**

1 Reading Guide

In class we have learned:

- How to encode objects using binary strings, and how this allows us to have a general definition of a computational problem. You should review this in §0.1 and §0.2 of the book.
- What is a Turing machine, and what is a Universal Turing machine. You should review this in §1.1, §1.2, §1.4, and §1.7 of the book.
- We have seen the first example of a lower-bound: the halting problem is uncomputable! You should review this in §1.5 of the book.

There were several details that we covered very quickly, and which **must** be looked at carefully, in order to properly understand what is going on. This kind of familiarity will be necessary to understand what's being said in class.

2 Exercises

1. Show that the following variants of a Turing machine all compute exactly the same set of functions (don't worry about how fast the simulation is):
   (a) A multi-tape Turing machine (like we taught in class).
   (b) A Turing machine with a single tape.
   (c) A Turing machine with a two-dimensional tape, i.e. a piece of paper divided into squares. The tape head can now move up, down, left, right, or stay in the same place.
   (d) A Turing machine with *random-access tapes*. A random-access tape is actually two tapes, a *memory* tape and an *address tape*. The address tape is a normal tape, where the tape head can move back and forth. But the Turing machine has a special state q_{jump} which, when entered, causes the tape head in the memory tape to "jump" to the position which is written in binary on the address tape (if the address tape does not have a valid number written in binary, nothing happens).
   (e) A Turing machine which only has the alphabet $\{0, 1\}$. (hint: encode the other symbols in binary)
2. Show that the following alternative kinds of machine compute exactly the same set of functions as the Turing machine:
   (a) A *stack machine* which has a finite number of states, and a finite number of "stacks". Each stack stores a sequence of symbols $(\sigma_1, \sigma_2, \ldots)$ and depending on the current state and the symbols σ_1 at the top of the stack, the machine can decide what the state at the next time step will be, and can decide to add ("push") a new symbol on top of the stack, or remove ("pop") the current symbol at the top of the stack. (hint: simulate a tape with two stacks)

(b) A *counter machine* which has a finite number of states, and a finite number of counters, each counter holding a natural number. At any point, depending on the current state, the machine jumps to a different state and executes one of the following instructions on one of the counters: (1) add or subtract 1 to the counter, (2) multiply the value in the counter by two (3) divide the value in the counter by two (4) read the last bit of the number (i.e. the value $\mod 2$), and go to one of two states depending on the answer. Hint: Show how to simulate a stack machine.

3. Show that the following problems are not computable:

   (a) Given an algorithm A for computing a sequence $a_1 = A(1), \ldots, a_n = A(n), \ldots$ of rational numbers, does $\sum_{i=1}^{\infty} a_i$ converge?

   (b) Given an algorithm A for computing the binary expansion of a real number $x = 0.A(1)A(2)A(3)\ldots \in [0,1]$, is $x = 1/2$?

   (c) Fix some enumeration $M_1, M_2, \ldots$ of Turing machines. Given a number n, how many Turing machines among $M_1, \ldots, M_n$ halt on input 0?

4. Suppose God came down to earth and offered you a magical device, called an *oracle*, that solves the Halting problem. Show that, using this oracle, you could find out the true answer[1] to the following open mathematical problems:

   (a) The Goldbach conjecture.

   (b) The twin primes conjecture.

   (c) The Collatz conjecture.

   (d) Find other examples that you could solve using the oracle.

   (e) Find an example which you cannot see how to solve!

5. Suppose again you have access to an oracle for the Halting problem. We have seen in class that we could use it to solve the Entschendungsproblem (input: a logical predicate, output: whether the predicate is in ZFC). Show that we could use the oracle to *find* a proof of any provable statement (i.e. not just decide whether the proof exists).

[1] Not the *provability in ZFC*, but rather the true answer, i.e. whether the statement is true or false.

3 Training questions

The tests/exam may contain similar exercises to the ones below.

In all exercises, don't build the Turing machine explicitly, just provide a high-level (yet careful) argument of how it can be done.

1. An n-vertex graph is called d-regular if every vertex has degree exactly d (i.e. has d incident edges).

   Show that, for any fixed natural number $d \geq 1$, the problem of deciding whether a given n-vertex graph is d regular is in P.

   Then show that the problem of deciding whether G is d-regular, when given an n-vertex graph G and a number $d \in \{1, \ldots, n-1\}$ as input (so d is not fixed), is also in P.

2. A k-clique in an n-vertex undirected graph $G = ([n], E)$ is a set $S \subseteq [n]$ of $|S| = k$ vertices of G such that any pair of vertices in S has an edge between them ($\forall_{a,b \in S, a \neq b} \{a, b\} \in E$).

   Show that, for any fixed natural number $k \geq 2$, the problem of deciding whether a given n-vertex graph G has a k-clique is in P.

   How about the problem of deciding whether G has a k-clique, when given both G and k as input (so k is not fixed)?

3. Consider the problem where we are given a list $X = (x_1, \ldots, x_n)$ of natural numbers, where each x_i is bounded as $0 \leq x_i \leq n^3$, and we wish to *sort* the list, i.e., output a list Y with the same numbers,[2] but sorted in ascending order. Show that this problem can be solved in time $O(n(\log n)^2)$ on a multitape Turing machine (hint: if you assume that you have two sorted lists, each with n numbers, how does one *merge* these two lists into a single sorted list all $2n$ numbers).

[2] Including multiplicities, i.e., if a number appears repeated k times in X, it should also appear k times in Y.